

AUTOMATED SYSTEM FOR EMPLOYEE ATTRITION PREDICTION

A MINI - PROJECT REPORT

Submitted by

TINU A V (211701057)

VARSHINI KARTHIKEYAN (211701058)

In partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND DESIGN



RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY : CHENNAI 600 025

NOVEMBER 2024

RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI – 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**Automated System for Employee Attrition Prediction**” is the bonafide work of “**TINU A V (211701057) & VARSHINI KARTHIKEYAN (211701058)**” who carried out the project work under my supervision.

SIGNATURE

Mr. S. Uma Maheswara Rao

HEAD OF THE DEPARTMENT

Associate Professor

Department of

Computer Science and Design

Rajalakshmi Engineering College

Chennai - 602105

SIGNATURE

Dr. P. Revathy, M.E., Ph.D.,

SUPERVISOR

Professor

Department of

Computer Science and Design

Rajalakshmi Engineering College

Chennai - 602105

Submitted to project viva-voce examination for the subject CD19P10
Foundations of Data Science held on _____

Internal Examiner

External Examiner

ABSTRACT

Employee attrition in an organization poses significant challenges, leading to increased recruitment costs, disrupted services, and loss of expertise. Identifying at-risk employees before they decide to leave is crucial but complex due to various influencing factors such as job satisfaction, work-life balance, performance, and compensation. This project aims to address this challenge by developing a machine learning model to predict employee attrition. The solution involves data preprocessing to clean and prepare the dataset, applying and evaluating various machine learning algorithms. By integrating the predictive model into HR systems, organizations can identify high-risk employees in real-time, enabling targeted retention strategies and fostering a more stable workforce.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our Chairman **Mr.S.Meganathan, B.E, F.I.E.**, our Vice Chairman **Mr. Abhay Shankar Meganathan, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) Thangam Meganathan, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S. N. Murugesan, M.E., Ph.D.**, our beloved Principal, for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Mr. Uma Maheswar Rao.**, Professor and Head of the Department of Computer Science and Design for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our project coordinator and supervisor, **Dr. P Revathy, M.E., Ph.D.**, Professor, Department of Computer Science and Design, Rajalakshmi Engineering College, for her valuable guidance and support throughout the course of the project.

TINU A V (211701057)

VARSHINI KARTHIKEYAN (211701058)

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	iii
	LIST OF FIGURES	vii
	LIST OF ABBREVIATIONS	ix
1	INTRODUCTION	1
2	DEFINING THE PROBLEM	2
	2.1 Overview of the Problem Statement	
	2.2 Objectives	
	2.3 System Flow	
3	DATASET DESCRIPTION	4
	3.1 Dataset Source	
	3.2 Dataset Size and Structure	
	3.3 Dataset Feature Description	
4	DATA CLEANING AND PREPROCESSING	9
	4.1 Data Loading	
	4.2 Initial Observation	
	4.3 Handling Missing Values	
	4.4 Outlier Detection and Treatment	
	4.5 Dropping Irrelevant and Inconsistent Data	
5	EXPLORATORY DATA ANALYSIS	16
	5.1 Count Plot Analysis	
	5.2 Correlation Analysis	
	5.3 Box Plot Analysis	

	5.4 Pair Plot Analysis	
	5.5 Cramer's V Analysis	
	5.6 Data Insights Description	
6	MODEL BUILDING	27
	6.1 Feature Selection	
	6.1.1 Chi-Square Test	
	6.1.2 Logistic Regression Model's p-values	
	6.1.3 Feature Importance using Random Forest	
	6.2 Data Partition	
	6.3 Model Selection and Justification	
	6.3.1 Logistic Regression	
	6.3.2 LDA	
	6.3.3 QDA	
	6.3.4 Decision Tree	
	6.3.5 Random Forest	
	6.4 Hyperparameter Tuning	
	6.5 Model Training	
7	MODEL EVALUATION AND OPTIMIZATION	39
	7.1 Model Evaluation	
	7.2 Model Prediction	
	7.3 Evaluation Metrics	
	7.4 Saving the Best Model	
	7.5 Using the Saved Model for Prediction	
8	RESULTS AND CONCLUSION	47
	8.1 Employee Attrition Prediction Insights	
	8.2 Final Recommendation	
	8.3 Conclusion	
	REFERENCES	48

LIST OF FIGURES

FIGURE NO.	NAME OF THE FIGURE	PAGE NO.
1.1	Data Flow Diagram	3
4.1	DataSet Information	10
4.2	Unique Values	11
4.3	DataSet Description – Numerical Values	12
4.4	DataSet Description – Categorical Values	12
4.5	Null Values	13
4.6	Outlier Visualization	14
5.1	EDA – Count Plot	17
5.2	EDA – Correlation Analysis	18
5.3	EDA – Box Plot	20
5.4	EDA – Pair Plot	21
5.5	EDA – Cramer’s V	23
6.1	Chi-Square Test	28
6.2	Logistic Regression	29
6.3	Feature Importance Using Random Forest	31
6.4	Training and Testing DataSet	33
6.5	Model Comparison	37
6.6	Model Fitting	38
6.7	Quadratic Discriminant Analysis	38
7.1	Best Model	39

7.2	Prediction On Test Data	40
7.3	Model Saved	41
7.4	Prediction Using the Saved Model	45
7.5	Confusion Matrix	46

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
HR	Human Resources
CSV	Comma Separated Values
EDA	Exploratory Data Analysis
P - Value	Probability Value
LDA	Linear Discriminant Analysis
QDA	Quadratic Discriminant Analysis
CV	Cross Validation

CHAPTER 1

INTRODUCTION

Employee attrition is a major concern for organizations, significantly impacting their operational efficiency, financial performance, and workforce stability. High turnover rates not only lead to increased costs for recruitment, onboarding, and training but also disrupt workflows and diminish organizational expertise. Retaining experienced talent has become increasingly challenging due to factors such as job dissatisfaction, work-life balance issues, limited career growth opportunities, and inadequate compensation. In this competitive business landscape, retaining skilled employees is critical to ensuring productivity and maintaining a competitive edge. This project aims to develop a machine learning-based predictive model to address the challenge of employee attrition. By analyzing a wide range of factors, including job satisfaction, performance metrics, work-life balance, and compensation, the model identifies employees who are at high risk of leaving the organization. This allows HR teams to implement proactive and targeted retention strategies, reducing turnover rates and maintaining team dynamics. Integrating the predictive model into HR systems provides real-time insights, enabling organizations to make timely decisions to address employee concerns. This solution not only helps minimize recruitment and training costs but also reduces operational disruptions, fosters workforce engagement, and enhances overall employee satisfaction. By leveraging advanced analytics, the project aims to create a more stable and committed workforce, ensuring long-term organizational success in an evolving business environment.

.

CHAPTER 2

DEFINING THE PROBLEM

2.1 OVERVIEW OF THE PROBLEM STATEMENT

Employee attrition is a pressing issue for organizations, significantly impacting operational efficiency and financial stability. The departure of employees not only results in increased recruitment and training costs but also disrupts workflow and diminishes organizational expertise. Retaining talent has become increasingly challenging due to the interplay of diverse factors such as job satisfaction, work-life balance, career growth opportunities, and compensation. Despite the importance of addressing attrition proactively, identifying employees at risk of leaving remains a complex task, primarily due to the dynamic nature of employee behavior and influencing factors. Organizations require a robust, data-driven solution to predict attrition and implement effective retention strategies. This project tackles the problem by developing a predictive machine learning model that analyzes various attributes contributing to employee attrition. The solution encompasses data preprocessing to ensure data quality, leveraging advanced algorithms for accurate predictions, and integrating the model into HR systems. By identifying high-risk employees early, organizations can adopt targeted interventions, reduce turnover rates, and cultivate a more engaged and stable workforce.

2.2 OBJECTIVES

The objective of this project is to assist organizations in addressing the challenge of employee attrition by developing a predictive machine learning model. The solution aims to help HR teams identify employees at risk of leaving, providing actionable insights to implement targeted retention strategies. By analyzing key factors such as job satisfaction, performance, and compensation, the

model enables organizations to proactively address employee concerns, foster workforce stability, and improve employee satisfaction. The project seeks to reduce recruitment and training costs, minimize operational disruptions, and maintain team dynamics by retaining experienced talent. Additionally, the integration of the predictive model into HR systems ensures real-time decision-making, enhancing the organization's ability to adapt to industry challenges and sustain productivity while creating a more engaged and committed workforce.

2.3 SYSTEM FLOW

The data flow diagram in Fig 1.1 outlines the process of predicting employee attrition using machine learning. It begins with inputting employee data, followed by preprocessing and feature selection. The best model is trained, evaluated, and used for predictions, with results driving HR actions to improve retention strategies.

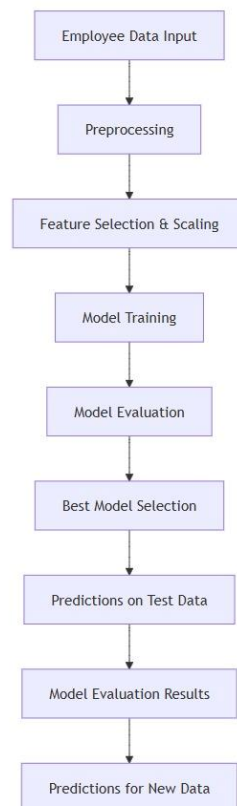


Fig 1.1 Data Flow Diagram

CHAPTER 3

DATASET DESCRIPTION

3.1 DATASET SOURCE:

The dataset used for this analysis is the Synthetic Employee Attrition Dataset, which was sourced from Kaggle. This simulated dataset is designed for the analysis and prediction of employee attrition. It provides a comprehensive set of features related to various aspects of employee profiles, such as demographics, job-related attributes, and personal circumstances. This resource is widely used for HR analytics and machine learning model development, providing a realistic view of the factors influencing employee retention and attrition. It is suitable for demonstrating advanced data analysis techniques and building predictive models to identify at-risk employees within an organization.

3.2 DATASET SIZE AND STRUCTURE:

The Synthetic Employee Attrition Dataset contains a total of 74,498 samples, which are typically split into training and testing sets. With an 80-20 split, the training set comprises 59,598 records, while the testing set includes 14,900 records. The dataset consists of 24 columns in total, with 9 numerical columns and 15 categorical columns.

- **Numerical Columns:** Age, Years at Company, Monthly Income, Number of Promotions, Number of Dependents, Distance from Home, Employee ID, Company Tenure, and the Attrition target variable

- **Categorical Columns:** Gender, Job Role, Work-Life Balance, Job Satisfaction, Performance Rating, Over Time, Education Level, Marital Status, Job Level, Company Size, Remote Work, Leadership Opportunities, Innovation Opportunities, Company Reputation, and Employee Recognition.

3.3 DATASET FEATURE DESCRIPTION

- **Employee ID**

A unique identifier assigned to each employee for distinction within the dataset. It consists of unique integers for every employee. This column is numerical.

- **Age**

Represents the employee's age, which helps analyze workforce demographics and identify trends related to career stage and attrition risk. Values range from 18 to 60 years. This column is numerical.

- **Gender**

Captures the gender of each employee, providing insights into gender distribution and its impact on attrition trends. Possible values include Male and Female. This column is categorical.

- **Years at Company**

Indicates the number of years the employee has been working at the company, offering insights into employee loyalty and potential attrition. Values are non-negative integers. This column is numerical.

- **Monthly Income**

Represents the monthly salary of the employee in dollars. It is an essential indicator of financial satisfaction and can influence attrition. Values are positive numeric values. This column is numerical.

- **Job Role**

Specifies the department or role of the employee, such as Finance, Healthcare, Technology, Education, and Media. This feature helps identify attrition patterns across different departments. This column is categorical.

- **Work-Life Balance**

Assesses how employees perceive the balance between their work and personal

life, an important factor affecting satisfaction and retention. Possible values are Poor, Below Average, Good, and Excellent. This column is categorical.

- **Job Satisfaction**

Measures the employee's satisfaction with their job, which directly impacts their likelihood of leaving. Possible values are Very Low, Low, Medium, and High. This column is categorical.

- **Performance Rating**

Reflects the employee's performance evaluation, helping identify trends between performance levels and attrition rates. Possible values include Low, Below Average, Average, and High. This column is categorical.

- **Number of Promotions**

Tracks the total number of promotions the employee has received during their tenure, reflecting career growth opportunities. Values are non-negative integers. This column is numerical.

- **Distance from Home**

Indicates the distance between the employee's home and workplace in miles. This factor can influence stress levels and attrition. Values are non-negative numeric values. This column is numerical.

- **Over Time**

Denotes whether the employee regularly works beyond standard hours. This can impact burnout and retention. Possible values are Yes and No. This column is categorical.

- **Education Level**

Represents the highest educational qualification attained by the employee, offering insights into skill levels. Possible values include High School, Associate Degree, Bachelor's Degree, Master's Degree, and PhD. This column is categorical.

- **Marital Status**

Highlights the employee's marital status, which may influence their career decisions and stability. Possible values are Divorced, Married, and Single. This column is categorical.

- **Number of Dependents**

Represents the number of individuals financially supported by the employee, such as children or other family members. Values are non-negative integers. This column is numerical.

- **Job Level**

Indicates the employee's hierarchical position, ranging from Entry to Senior levels. This helps analyze attrition across career stages. This column is categorical.

- **Company Size**

Represents the size of the organization the employee works for, categorized as Small, Medium, or Large. Company size can influence job experience and turnover rates. This column is categorical.

- **Company Tenure**

Captures the total years the employee has worked in the industry. It provides insights into career stability and experience. Values are non-negative integers. This column is numerical.

- **Remote Work**

Indicates whether the employee works remotely. Remote work flexibility can impact satisfaction and attrition. Possible values are Yes and No. This column is categorical.

- **Leadership Opportunities**

Denotes whether the employee has access to leadership roles, which can enhance satisfaction and retention. Possible values are Yes and No. This column is categorical.

- **Innovation Opportunities**

Reflects whether employees are encouraged to engage in innovation, which can influence engagement and retention. Possible values are Yes and No. This column is categorical.

- **Company Reputation**

Gauges the employee's perception of the company's reputation. A good reputation can boost loyalty. Possible values are Very Poor, Poor, Good, and Excellent. This column is categorical.

- **Employee Recognition**

Assesses the level of recognition employees receive for their contributions, which impacts motivation and retention. Possible values are Very Low, Low, Medium, and High. This column is categorical.

- **Attrition**

The target variable indicating whether the employee has left the company, with values encoded as 0 (stayed) and 1 (left). This column is numerical.

CHAPTER 4

DATA CLEANING AND PREPROCESSING

4.1 DATA LOADING:

In this project, the data is loaded using the pandas library. The training and testing datasets are separately loaded from CSV files using `pd.read_csv()`. The datasets are then merged using `pd.concat()` with `ignore_index=True` to ensure they are treated as a single DataFrame, enabling seamless analysis. This approach allows for consistent preprocessing, where missing values are handled, categorical variables are encoded, and numerical features are standardized to prepare the data for model development and evaluation.

Python Code to Load Data Set:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import warnings
from sklearn.model_selection import train_test_split, cross_val_score,
GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
recall_score, f1_score, make_scorer
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
QuadraticDiscriminantAnalysis
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
import statsmodels.api as sm
from scipy.stats import chi2_contingency
warnings.filterwarnings('ignore')
train = pd.read_csv(r"/train.csv")
test = pd.read_csv(r"/test.csv")
```

4.2 INITIAL OBSERVATIONS:

Upon loading the dataset, initial observations are made regarding its structure and the features as in Fig 4.1 included, such as demographic details, job-related attributes, and employee behaviors. Fig 4.2 examines unique values in each column. Fig 4.3 and Fig 4.4 describes the statistics of numerical and categorical columns. The script identifies and processes any missing values and examines the distribution of key features like Age, Monthly Income, Job Role, and Job Satisfaction. New features, such as Work-Life Balance and Employee Recognition, are explored to assess their influence on employee attrition. Correlations between these features are analyzed, and any patterns indicating the potential predictors for employee turnover are noted. These insights guide further data cleaning, feature engineering, and model development.

Python Code for data information:

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 74498 entries, 0 to 74497
Data columns (total 24 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Employee ID                          74498 non-null  int64
1   Age                                  74498 non-null  int64
2   Gender                              74498 non-null  object
3   Years at Company                    74498 non-null  int64
4   Job Role                            74498 non-null  object
5   Monthly Income                      74498 non-null  int64
6   Work-Life Balance                   74498 non-null  object
7   Job Satisfaction                    74498 non-null  object
8   Performance Rating                  74498 non-null  object
9   Number of Promotions                74498 non-null  int64
10  Overtime                            74498 non-null  object
11  Distance from Home                  74498 non-null  int64
12  Education Level                     74498 non-null  object
13  Marital Status                      74498 non-null  object
14  Number of Dependents                74498 non-null  int64
15  Job Level                           74498 non-null  object
16  Company Size                        74498 non-null  object
17  Company Tenure                      74498 non-null  int64
18  Remote Work                         74498 non-null  object
19  Leadership Opportunities            74498 non-null  object
20  Innovation Opportunities            74498 non-null  object
21  Company Reputation                  74498 non-null  object
22  Employee Recognition                74498 non-null  object
23  Attrition                           74498 non-null  object
dtypes: int64(8), object(16)
```

Fig 4.1 Dataset Information

Unique Values in each column

```
cols = df.columns.tolist()
```

```
for col in cols:
```

```
    print(f'{col}: has {df[col].nunique()} unique values.')
```

```
    print(df[col].unique())
```

```
    print("---"*25)
```

```
Employee ID: has 74498 unique values.
[ 8410 64756 30257 ... 12409 9554 73042]
-----
Age: has 42 unique values.
[31 59 24 36 56 38 47 48 57 30 29 40 19 33 49 51 39 54 23 45 42 53 37 34
 25 41 55 21 28 26 52 50 27 58 43 35 46 44 20 18 22 32]
-----
Gender: has 2 unique values.
['Male' 'Female']
-----
Years at Company: has 51 unique values.
[19  4 10  7 41  3 23 16 44  1 12  6 38 22 30  9 37 33 13  5 21 32 34 11
  2 18 27 17 36  8 28 35 14 24 40 15 45 39 20 43 26 29 46 42 25 31 47 48
 49 50 51]
-----
Job Role: has 5 unique values.
['Education' 'Media' 'Healthcare' 'Technology' 'Finance']
-----
Monthly Income: has 9842 unique values.
[ 5390 5534 8159 ... 11854 11558 12651]
-----
Work-Life Balance: has 4 unique values.
['Excellent' 'Poor' 'Good' 'Fair']
-----
Job Satisfaction: has 4 unique values.
['Medium' 'High' 'Very High' 'Low']
-----
Performance Rating: has 4 unique values.
['Average' 'Low' 'High' 'Below Average']
-----
Number of Promotions: has 5 unique values.
[2 3 0 1 4]
-----
Overtime: has 2 unique values.
['No' 'Yes']
-----
Distance from Home: has 99 unique values.
[22 21 11 27 71 37 75  5 39 57 51 26 78 30 98 48 17 86 60 10 18 95 23 16
 63 92 58 55 31 32 82  8 74  3 76 47 67 84 14 36 79 52 44 89 64  7 87 24
 15 81 13 35 68 56 73 41 34 29 19 50 62 49 69 33 61 53 72 91 65 93 28 46
 54  9 45 96 94  1 88 25  6 70 12 99  4 97 42 38 90 43 59 85  2 40 20 77
 83 80 66]
-----
Education Level: has 5 unique values.
['Associate Degree' 'Master's Degree' 'Bachelor's Degree' 'High School'
 'PhD']
-----
Marital Status: has 3 unique values.
['Married' 'Divorced' 'Single']
-----
Number of Dependents: has 7 unique values.
[0 3 2 4 1 5 6]
-----
Job Level: has 3 unique values.
['Mid' 'Senior' 'Entry']
-----
Company Size: has 3 unique values.
['Medium' 'Small' 'Large']
-----
-----
Company Tenure: has 127 unique values.
[ 89  21  74  50  68  47  93  88  75  45  17  38  84  96  63  19  70  76
 72  95  20  40  60  23  83  67  91  28  27  55  54  71  46  86  69  36
 22  25  49  61 103  64  12  39  33  57  58  65  66 100  62  80  6  78
 30  77  87  41  97  31  34  14  35  16  81  52 106  90  53  73  37  5
 94 119  56  15  10 118  32  59  82  9  11  44  26  24  79  92 117  48
 29  18  85  13 102  8 107 114  42  43  51 101  7  98  4 109  2 110
111 108 123 105  99 113 124  3 112 104 115 116 120 125 127 121 122 126
128]
-----
Remote Work: has 2 unique values.
['No' 'Yes']
-----
Leadership Opportunities: has 2 unique values.
['No' 'Yes']
-----
Innovation Opportunities: has 2 unique values.
['No' 'Yes']
-----
Company Reputation: has 4 unique values.
['Excellent' 'Fair' 'Poor' 'Good']
-----
Employee Recognition: has 4 unique values.
['Medium' 'Low' 'High' 'Very High']
-----
Attrition: has 2 unique values.
['Stayed' 'Left']
-----
```

Fig 4.2 Unique values

Summary statistics of numerical columns

```
df.describe().drop('Employee ID', axis=1).T
```

	count	mean	std	min	25%	50%	75%	max
Age	74498.0	38.529746	12.083456	18.0	28.0	39.0	49.0	59.0
Years at Company	74498.0	15.721603	11.223744	1.0	7.0	13.0	23.0	51.0
Monthly Income	74498.0	7299.379514	2152.508566	1226.0	5652.0	7348.0	8876.0	16149.0
Number of Promotions	74498.0	0.832935	0.995289	0.0	0.0	1.0	2.0	4.0
Distance from Home	74498.0	49.991584	28.513611	1.0	25.0	50.0	75.0	99.0
Number of Dependents	74498.0	1.650326	1.553633	0.0	0.0	1.0	3.0	6.0
Company Tenure	74498.0	55.727456	25.399349	2.0	36.0	56.0	76.0	128.0

Fig 4.3 Dataset Description- Numerical columns

Summary statistics of categorical columns

`df.describe(include='object').T`

	count	unique	top	freq
Gender	74498	2	Male	40826
Job Role	74498	5	Technology	19322
Work-Life Balance	74498	4	Good	28158
Job Satisfaction	74498	4	High	37245
Performance Rating	74498	4	Average	44719
Overtime	74498	2	No	50157
Education Level	74498	5	Bachelor's Degree	22331
Marital Status	74498	3	Married	37419
Job Level	74498	3	Entry	29780
Company Size	74498	3	Medium	37231
Remote Work	74498	2	No	60300
Leadership Opportunities	74498	2	No	70845
Innovation Opportunities	74498	2	No	62394
Company Reputation	74498	4	Good	37182
Employee Recognition	74498	4	Low	29620
Attrition	74498	2	Stayed	39128

Fig 4.4 Dataset Description- Categorical columns

4.3 HANDLING MISSING VALUES

The dataset is first examined for missing values as shown in Fig 4.5 . Numerical features are handled by imputing missing data with the mean or median, depending on the distribution of the data. For categorical variables like Job Role or Department, mode imputation is applied to replace missing values with the most frequent category. Columns with excessive missing values or those deemed irrelevant, such as Employee ID, are dropped from the dataset to ensure high-quality data for modeling.

Python Code to Handle Missing Values:

```
df.duplicated().sum()
```

```
df.isnull().sum()
```

Employee ID	0
Age	0
Gender	0
Years at Company	0
Job Role	0
Monthly Income	0
Work-Life Balance	0
Job Satisfaction	0
Performance Rating	0
Number of Promotions	0
Overtime	0
Distance from Home	0
Education Level	0
Marital Status	0
Number of Dependents	0
Job Level	0
Company Size	0
Company Tenure	0
Remote Work	0
Leadership Opportunities	0
Innovation Opportunities	0
Company Reputation	0
Employee Recognition	0
Attrition	0

Fig 4.5 Null Values

4.4 OUTLIER DETECTION AND TREATMENT

Outliers are identified in key numerical features such as Monthly Income, Age, and Years at Company. Techniques like box plots in Fig 4.6 and Z-scores are used to detect extreme values that could skew the analysis. For example, employees with unusually high salaries or an unreasonably high number of years in the company may be considered outliers. These outliers are treated either by capping the values to a reasonable range or by removing the rows entirely to prevent them from distorting the analysis and model predictions.

Python Code for Outlier Detection:

```
# List of columns for which to create box plots
numerical_columns = [
    'Age', 'Years at Company', 'Monthly Income', 'Distance from Home',
    'Number of Promotions', 'Company Tenure']
# Set up the matplotlib figure with 2 rows and 2 columns for subplots
fig, axes = plt.subplots(2, 3, figsize=(14, 10))
# Generate box plots for each column in a separate subplot
for ax, col in zip(axes.flatten(), numerical_columns):
    sns.boxplot(data=df, y=col, ax=ax)
    ax.set_title(f'Box Plot of {col}')
# Display the plots
plt.show()
```

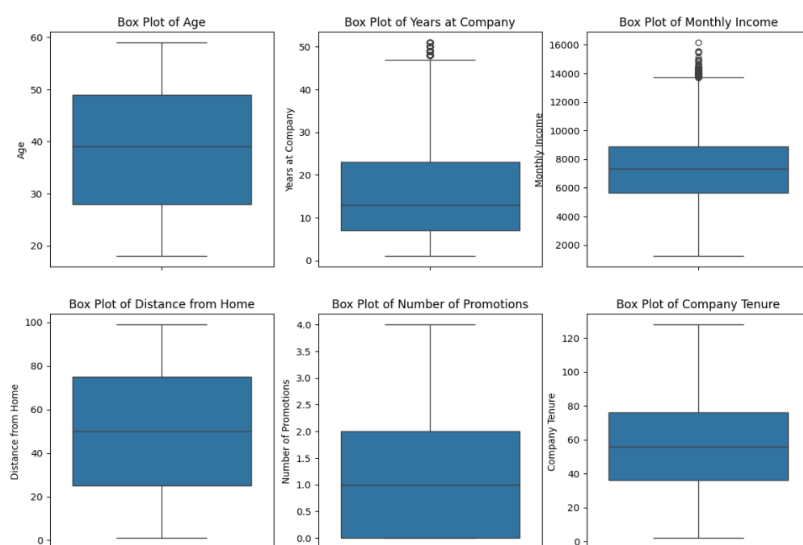


Fig 4.6 Outlier Visualization

4.5 DROPPING IRRELEVANT OR INCONSISTENT DATA

Inconsistencies in the dataset are identified and addressed by dropping rows with data entry errors. A specific example includes cases where the Age and Years at Company columns have unrealistic discrepancies, such as an employee being 18 years old with 51 years of experience. These rows are removed to maintain the integrity of the dataset. A total of 29,812 rows, where the absolute difference between Age and Years at Company is less than 18, are dropped. This step ensures that the data used for analysis and modeling is both consistent and realistic.

Python Code Dropping irrelevant Data:

```
# All such rows where absolute value of 'Age' - 'Years at Company' is less than 18
df[abs(df['Age'] - df['Years at Company']) < 18].sample(10)
# Count of all such rows
df[abs(df['Age'] - df['Years at Company']) < 18].shape[0]
# Dropping all such rows
df = df[abs(df['Age'] - df['Years at Company']) >= 18]
# Dropping the Job Role column
df.drop('Job Role', axis=1, inplace=True)
df.head()
```


CHAPTER 5

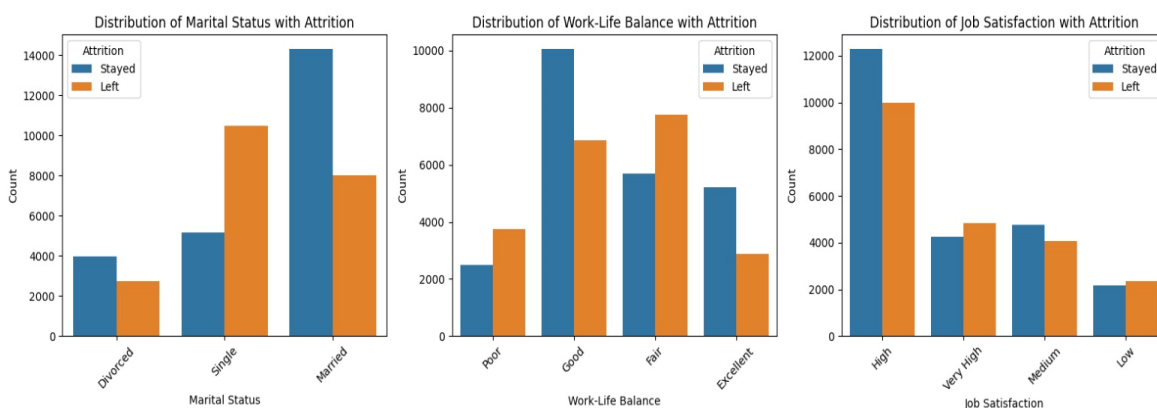
EXPLORATORY DATA ANALYSIS

5.1 COUNT PLOT ANALYSIS

Count plot in Fig 5.1 is used to visualize the distribution of categorical variables, such as Gender, Job Satisfaction, and Work-Life Balance, against employee attrition. The plot helps in understanding how each category within these variables relates to the likelihood of attrition. By plotting Attrition as the hue, it provides a clear visual comparison of how different categories influence whether employees leave or stay.

Python Code for Count Plot:

```
# List of categorical columns to analyze
categorical_columns = ['Marital Status', 'Work-Life Balance', 'Job Satisfaction',
'Performance Rating','Overtime', 'Remote Work', 'Leadership Opportunities',
'Innovation Opportunities', 'Company Reputation', 'Employee Recognition',
'Attrition']
plt.figure(figsize=(16, 20))
for i, col in enumerate(categorical_columns, 1):
    plt.subplot(4, 3, i)
    sns.countplot(data=df, x=col, hue='Attrition')
    plt.title(f'Distribution of {col} with Attrition')
    plt.xlabel(col)
    plt.xticks(rotation=45)
    plt.ylabel('Count')
plt.tight_layout()
plt.show()
```



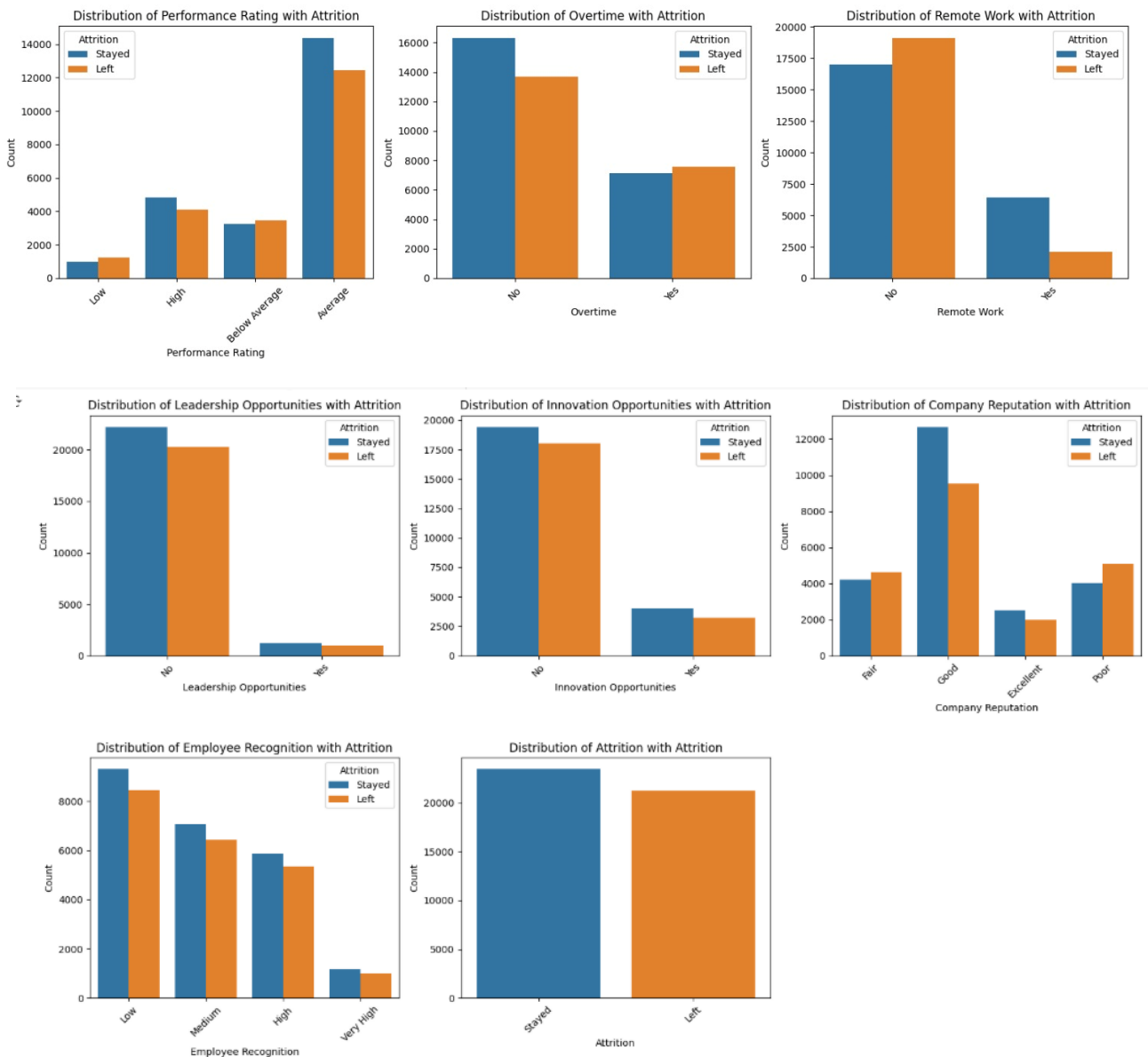


Fig 5.1 EDA-Count Plot

INSIGHTS

- Gender: Males tend to stay longer compared to females.
- Work-Life Balance: Poor work-life balance increases the likelihood of attrition.
- Job Satisfaction: Employees with very low or very high satisfaction tend to leave, while those with medium to high satisfaction stay longer.
- Overtime: Employees who work overtime are more likely to leave if not properly compensated.
- Education: There is no strong correlation between education level and attrition.

5.2 CORRELATION ANALYSIS

In this section, we analyze the correlations between numerical variables like Age, Monthly Income, Years at Company, and others. A heatmap in Fig 5.2 is used to visualize these relationships, showing the degree of association between pairs of variables.

Python Code for Correlation Analysis:

```
# List of numerical columns to analyze
numerical_columns = ['Age', 'Years at Company', 'Monthly Income', 'Number of
Promotions', 'Distance from Home', 'Number of Dependents', 'Company Tenure']
# Heatmap to show correlation between numerical variables
plt.figure(figsize=(10, 8))
sns.heatmap(df[numerical_columns].corr(), annot=True, cmap='coolwarm',
fmt='.2f')
plt.title('Correlation Heatmap of Numerical Variables')
plt.show()
```

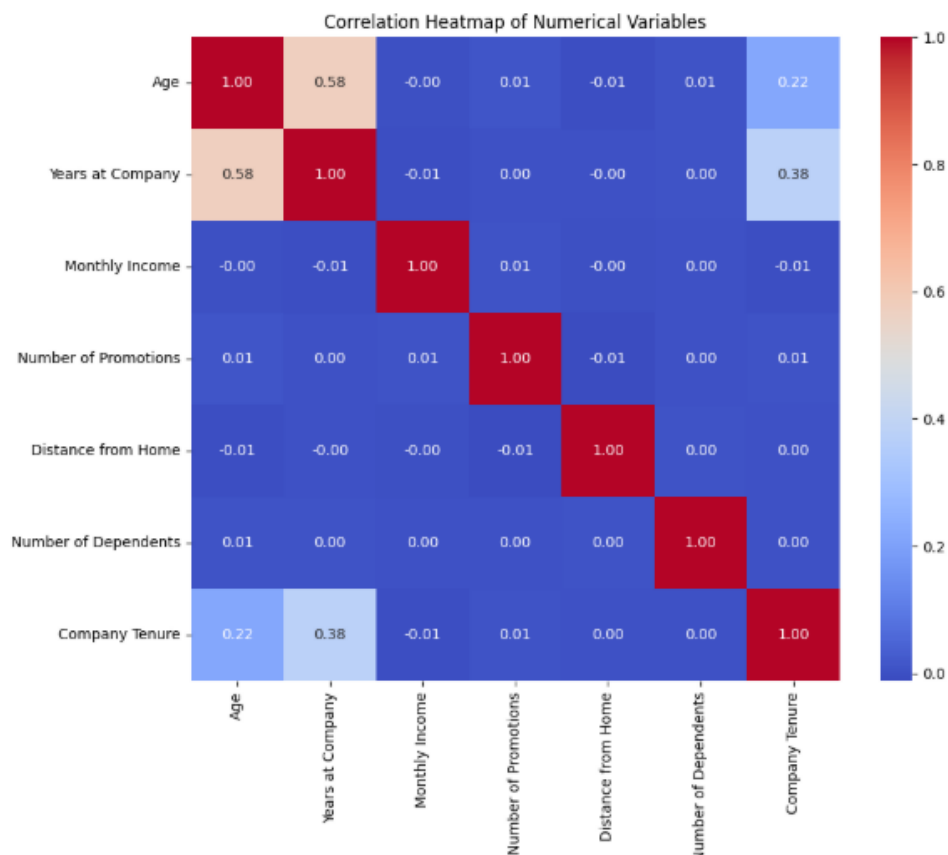


Fig 5.2 EDA-Correlation Analysis

INSIGHTS

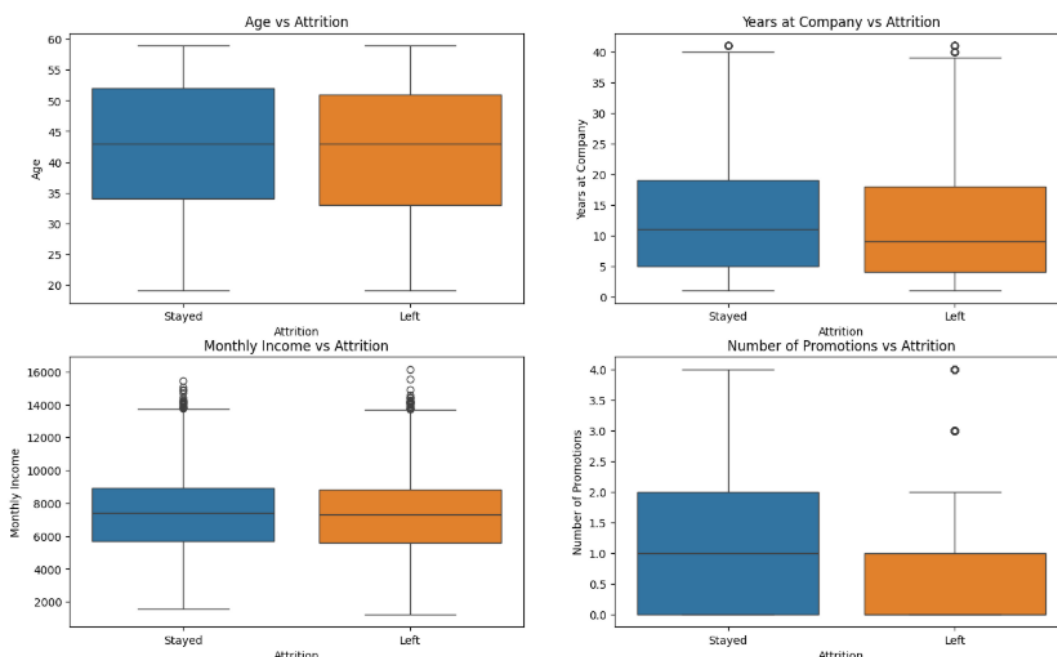
- Age vs. Years at Company: A moderate positive correlation suggests that older employees typically have longer tenure at the company.
- Age vs. Company Tenure & Years at Company vs. Company Tenure: These variables show slight correlations, indicating that while they are related, factors such as job changes prevent a perfect correlation.

5.3 BOXPLOT ANALYSIS

Boxplots in Fig 5.3 are used to analyze the distribution of numerical features, such as Age, Monthly Income, Number of Promotions, Distance from Home, and Company Tenure, against Attrition. These plots provide a visual understanding of how these variables relate to whether employees stay or leave.

Python Code for Box Plot:

```
plt.figure(figsize=(16, 20))
for i,col in enumerate(numerical_columns,1):
    plt.subplot(4,2,i)
    sns.boxplot(x='Attrition', y=col, data=df,hue='Attrition')
    plt.title(f'{col} vs Attrition')
plt.show()
```



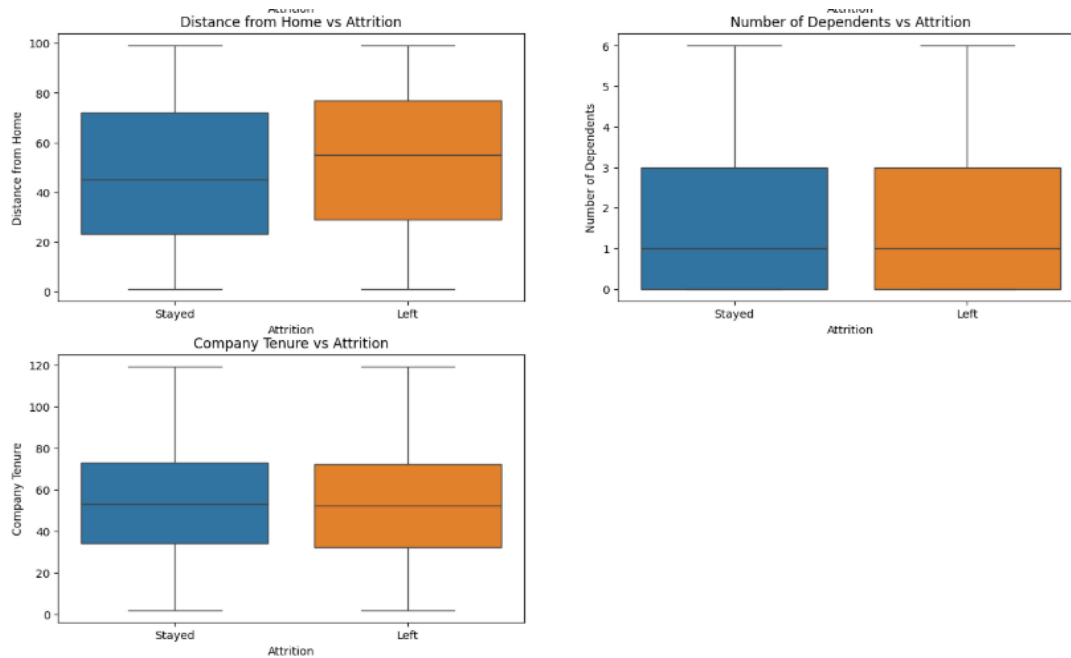


Fig 5.3 EDA-Box Plot

INSIGHTS

- Monthly Income: No significant difference between employees who stay vs. leave, though high-income outliers could reflect senior roles.
- Distance from Home: No clear pattern emerges, but it could indirectly influence attrition when combined with other factors.
- Age: Older employees tend to stay longer, suggesting job stability with age.
- Number of Dependents: Employees with more dependents tend to stay longer, possibly due to fewer mobility options.
- Company Tenure: Longer-tenured employees are more likely to stay.

5.4 PAIR PLOT ANALYSIS

A pair plot in Fig 5.4 allows for the visualization of relationships between multiple numerical variables and their correlation with attrition. It helps in identifying patterns and trends across various features. The scatter plots reveal that most numerical features like Age, Monthly Income, and Distance from Home do not show strong interdependencies.

Python Code for Pair Plot:

```
sns.pairplot(df, hue='Attrition', vars=numerical_columns)
plt.show()
```

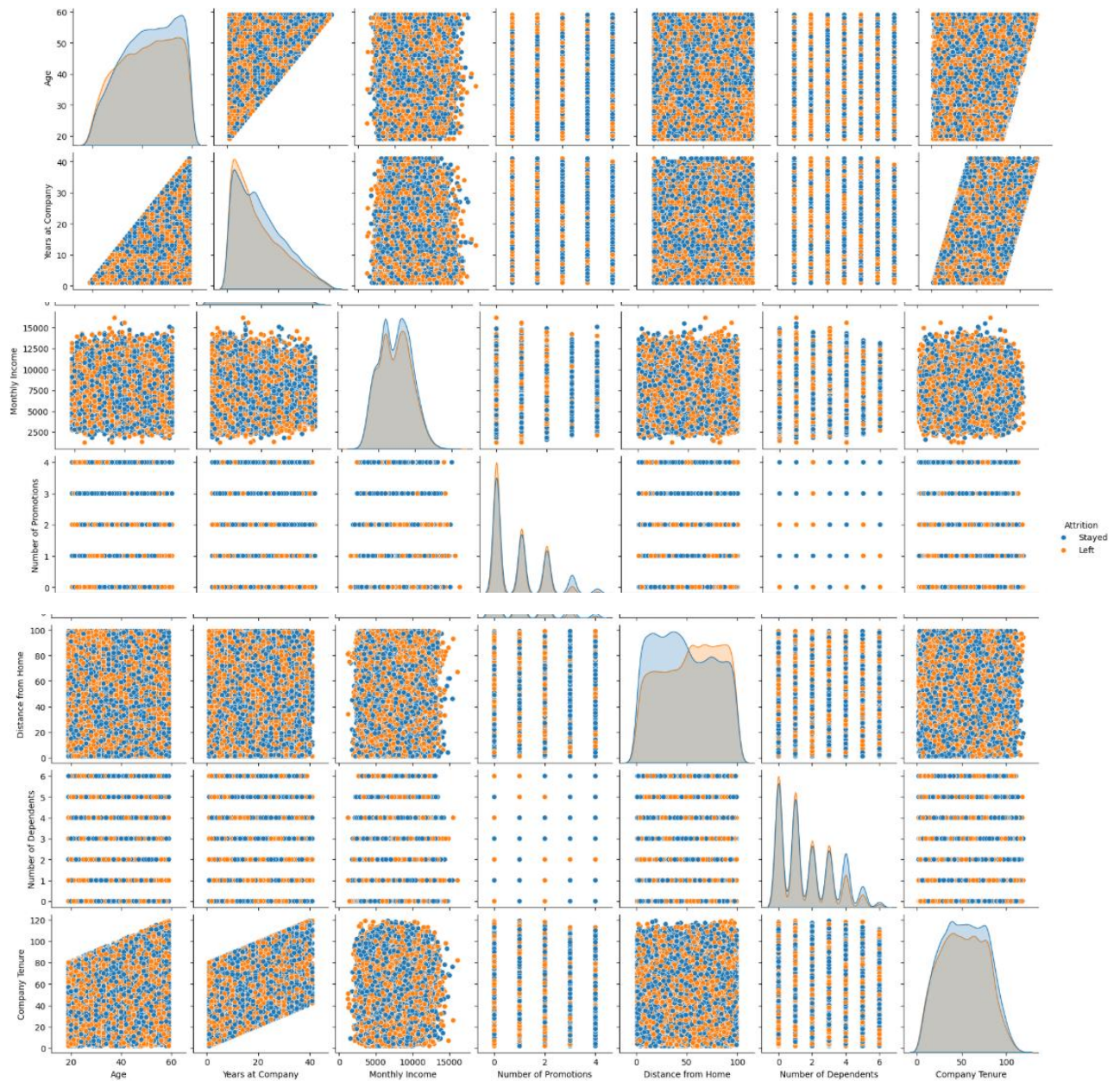


Fig 5.4 EDA-Pair Plot

INSIGHTS

- There is a moderate positive correlation between Age and Years at Company, suggesting that older employees generally have more company tenure.

- Promotion counts show a significant trend: employees with more promotions (around 3 to 4) are more likely to stay. This suggests that employees who experience growth in their roles feel more valued and engaged, reducing their likelihood of leaving.
- The histograms on the diagonal show balanced distributions for both "Stayed" and "Left" groups, indicating that no single variable dominates attrition decisions. This reinforces the idea that multiple factors influence whether an employee stays or leaves the company.

5.5 CRAMER'S ANALYSIS

Cramér's V is a measure of association for nominal (categorical) variables. It quantifies the strength of association between two categorical variables, ranging from 0 (no association) to 1 (perfect association). It is based on the Chi-square statistic and adjusts for the number of categories in the variables. In Fig 5.5, higher Cramér's V value indicates a stronger association, while lower values suggest weak or no relationship. This metric is useful when visualizing relationships between categorical variables, helping to identify potential dependencies that could influence analysis or predictive models.

Python Code for Cramer's V:

```
# Function to calculate Cramér's V
def cramers_v(var1, var2):
    crosstab = pd.crosstab(var1, var2) # Cross table
    chi2 = chi2_contingency(crosstab)[0]
    n = crosstab.sum().sum() # Sample size
    r, k = crosstab.shape
    return np.sqrt(chi2 / (n * (min(r, k) - 1)))

# Apply to pairs of categorical columns
categorical_columns = ['Gender', 'Work-Life Balance', 'Job Satisfaction',
'Performance Rating',
'Overtime', 'Education Level', 'Marital Status', 'Remote Work',
'Leadership Opportunities', 'Innovation Opportunities', 'Company
Reputation', 'Employee Recognition']
```



```

cramers_v_matrix = pd.DataFrame(index=categorical_columns,
columns=categorical_columns)
for col1 in categorical_columns:
    for col2 in categorical_columns:
        crammers_v_matrix.loc[col1, col2] = crammers_v(df[col1], df[col2])
# Convert to numeric for a heatmap visualization
cramers_v_matrix = crammers_v_matrix.astype(float)
# Plot the heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(cramers_v_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Cramér\'s V Heatmap for Categorical Variables')
plt.show()

```

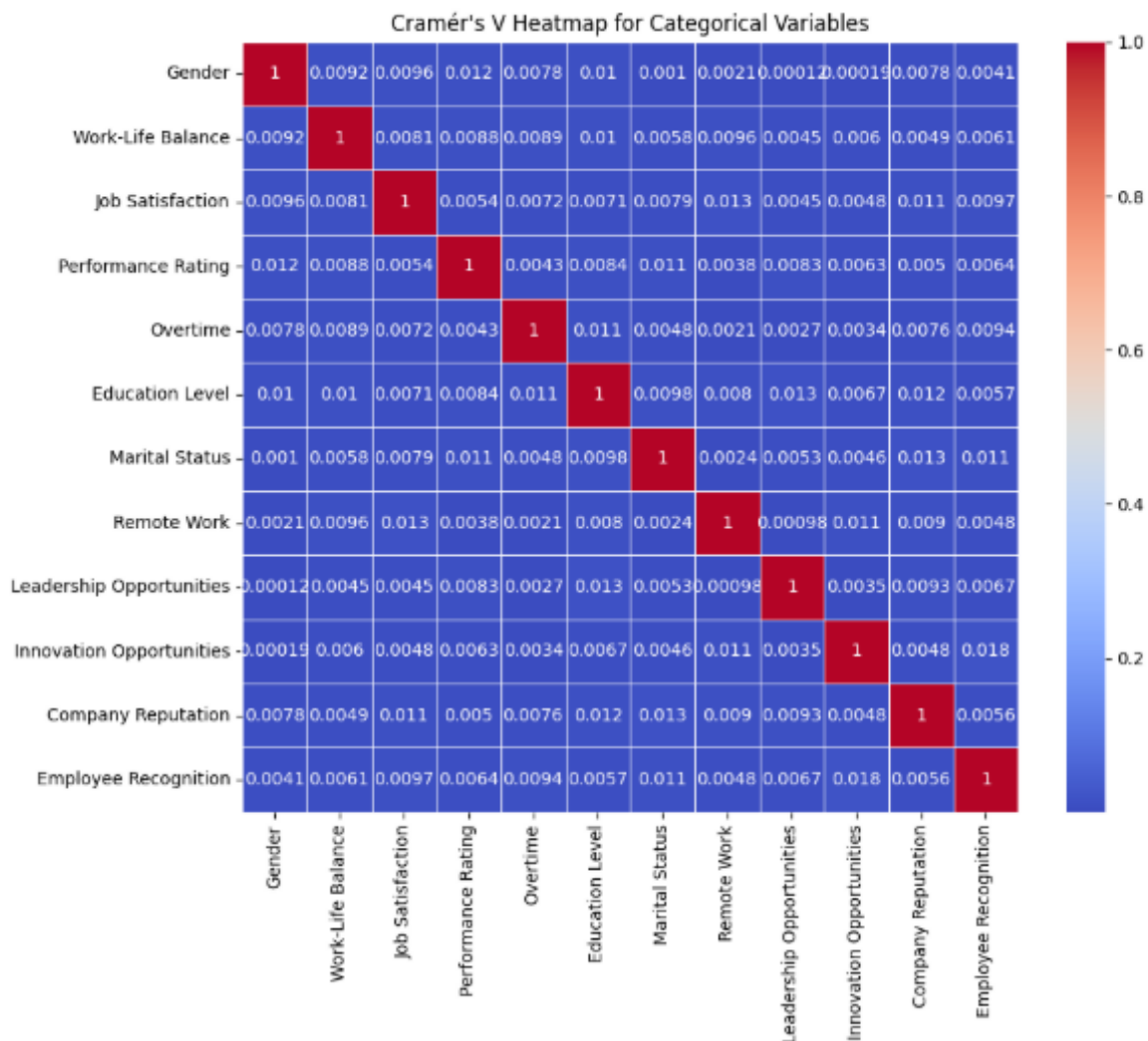


Fig 5.5 EDA- Cramer's V

INSIGHTS

- Weak correlations between most categorical variables suggest they don't have strong interdependencies.
- Job Satisfaction shows a weak relationship with Remote Work, indicating that employees with remote work options may have slightly higher satisfaction.
- Education Level is weakly related to Leadership Opportunities and Company Reputation, suggesting that more educated employees might value leadership roles or company reputation more.
- Company Reputation and Employee Recognition show weak associations with Education Level and Marital Status, indicating that employees' personal backgrounds and education may slightly influence their perceptions of company reputation and recognition.

5.6 DATA INSIGHTS DESCRIPTION

S.NO	DATA INSIGHT	DESCRIPTION
1.	Age vs. Attrition	Older employees with longer tenures are less likely to leave, while younger employees may have higher turnover rates.
2.	Work-Life Balance vs. Attrition	Poor work-life balance correlates with higher attrition, suggesting employees who feel overworked are more likely to leave.
3.	Job Satisfaction vs. Attrition	Employees with low satisfaction are more likely to leave, while those with high satisfaction tend to stay longer.

4.	Monthly Income vs. Attrition	Income is not a strong predictor of attrition, but very low or very high salaries may have some impact.
5.	Number of Promotions vs. Attrition	Employees with fewer promotions are more likely to leave, indicating promotions are a key retention factor.
6.	Years at Company vs. Attrition	Employees with longer tenures at the company tend to stay longer, showing that loyalty builds over time.
7.	Job Level vs. Attrition	Senior employees are less likely to leave, likely due to higher job security, perks, and career stability.
8.	Remote Work vs. Attrition	Employees who work remotely may have higher job satisfaction and lower attrition rates, aligning with current work trends.
9.	Marital Status vs. Attrition	Single employees tend to have higher attrition rates, possibly because they have fewer personal commitments and more flexibility to switch jobs.
10.	Leadership Opportunities vs. Attrition	Employees without leadership opportunities are more likely to leave, suggesting that growth and advancement are important retention factors.
11.	Company Reputation vs. Attrition	Employees who perceive the company positively are less likely to leave, highlighting the role of company reputation in retention.

12.	Education Level vs. Attrition	Higher education levels correlate with lower attrition, as more educated employees may feel more secure or satisfied in their roles.
13.	Innovation Opportunities vs. Attrition	Employees with access to innovation opportunities tend to stay longer, possibly because they feel more valued and challenged in their roles.
14.	Employee Recognition vs. Attrition	A higher level of recognition decreases the likelihood of attrition, as employees who feel valued are more likely to remain.

CHAPTER 6

MODEL BUILDING

6.1 FEATURE SELECTION

Feature Selection is a crucial step in the machine learning pipeline, where relevant features are identified and irrelevant or redundant ones are removed to enhance model performance. It reduces dimensionality, thereby improving computational efficiency and reducing the risk of overfitting. Effective feature selection can uncover the most significant predictors, making models more interpretable and robust.

6.1.1 CHI-SQUARE TEST FOR CATEGORICAL VARIABLES

The chi-square test results in Fig 6.1 highlight which categorical features have a statistically significant relationship with employee attrition. Gender, Work-Life Balance, Job Satisfaction, Performance Rating, Overtime, Education Level, Marital Status, Remote Work, and Company Reputation all exhibit strong associations with attrition, making them key predictors for modeling attrition risk. These features are likely to provide valuable insights when building predictive models.

Conversely, features such as Leadership Opportunities, Innovation Opportunities, and Employee Recognition demonstrate weak significance with attrition, indicating that they have little impact on employee turnover decisions. While they might provide some insight into the work environment, these variables are not as critical when predicting attrition. Thus, they may be considered for exclusion from the model to enhance its accuracy and reduce complexity.

This feature selection process helps narrow down the most impactful factors for predicting attrition, leading to more efficient model training. By focusing on variables with strong relationships to attrition, we can build a more reliable predictive model that supports targeted retention strategies.

Python Code for Chi-Square Test:

```
for col in categorical_columns:
    contingency_table = pd.crosstab(df[col], df['Attrition'])
    chi2, p, dof, expected = chi2_contingency(contingency_table)
    print(f'{col}: Chi-Square Statistic = {chi2}, p-value = {p}')

% Gender: Chi-Square Statistic = 480.9090672111272, p-value = 1.3549805541186008e-106
% Work-Life Balance: Chi-Square Statistic = 1755.0775839009605, p-value = 0.0
% Job Satisfaction: Chi-Square Statistic = 224.26062352662842, p-value = 2.4080455257450425e-48
% Performance Rating: Chi-Square Statistic = 126.74414258468278, p-value = 2.72050811750671e-27
% Overtime: Chi-Square Statistic = 136.34511654539483, p-value = 1.6769709095148664e-31
% Education Level: Chi-Square Statistic = 532.4531271002018, p-value = 6.3995898503256964e-114
% Marital Status: Chi-Square Statistic = 3712.1045773426217, p-value = 0.0
% Remote Work: Chi-Square Statistic = 2216.6394911687203, p-value = 0.0
% Leadership Opportunities: Chi-Square Statistic = 8.950631397249657, p-value = 0.0027737369896921977
% Innovation Opportunities: Chi-Square Statistic = 28.761877462625982, p-value = 8.184642236642496e-08
% Company Reputation: Chi-Square Statistic = 528.4329382249655, p-value = 3.2847322263609506e-114
% Employee Recognition: Chi-Square Statistic = 2.1062103120625224, p-value = 0.5506573970263993
```

Fig 6.1 Chi-Square Test

6.1.2 LOGISTIC REGRESSION MODEL'S P-VALUES FOR NUMERICAL VARIABLES

In logistic regression, the dependent variable, Attrition, is converted to a binary format where "1" represents attrition (employee left) and "0" represents retention (employee stayed). This transformation allows the model to predict the probability of attrition based on numerical predictors like Age, Years at Company, Monthly Income, Number of Promotions, Distance from Home, Number of Dependents, and Company Tenure.

Python Code for Logistic Regression:

```
# Define the independent variables (numerical columns) and the dependent variable (Attrition)
numerical_columns = ['Age', 'Years at Company', 'Monthly Income', 'Number of Promotions', 'Distance from Home', 'Number of Dependents', 'Company Tenure']
X = df[numerical_columns]
y = df['Attrition'].apply(lambda x: 1 if x == 'Left' else 0) # Convert Attrition to 0 and 1

# Add a constant to the model (intercept)
X = sm.add_constant(X)
```

```
# Fit the logistic regression model
logit_model = sm.Logit(y, X)
result = logit_model.fit()

# Print the summary to see p-values and other statistics
print(result.summary())
```

Logit Regression Results						
Dep. Variable:	Attrition	No. Observations:	44686			
Model:	Logit	Df Residuals:	44678			
Method:	MLE	Df Model:	7			
Date:	Thu, 14 Nov 2024	Pseudo R-squ.:	inf			
Time:	14:14:47	Log-Likelihood:	-6.5033e-11			
converged:	False	LL-Null:	0.0000			
Covariance Type:	nonrobust	LLR p-value:	1.000			
	coef	std err	z	P> z	[0.025	0.975]
const	-25.7139	7.61e+05	-3.38e-05	1.000	-1.49e+06	1.49e+06
Age	-0.0393	1.67e+04	-2.35e-06	1.000	-3.28e+04	3.28e+04
Years at Company	0.0662	1.9e+04	3.49e-06	1.000	-3.72e+04	3.72e+04
Monthly Income	-0.0014	114.671	-1.23e-05	1.000	-224.752	224.749
Number of Promotions	0.0112	1.25e+05	9e-08	1.000	-2.45e+05	2.45e+05
Distance from Home	0.0094	4455.228	2.11e-06	1.000	-8732.077	8732.095
Number of Dependents	-0.0448	8.05e+04	-5.57e-07	1.000	-1.58e+05	1.58e+05
Company Tenure	-0.0215	5731.568	-3.75e-06	1.000	-1.12e+04	1.12e+04

Complete Separation: The results show that there is complete separation or perfect prediction. In this case the Maximum Likelihood Estimator does not exist and the parameters are not identified.

Fig 6.2 Logistic Regression

INSIGHTS

The model in Fig 6.2 highlights Years at Company and Number of Promotions as significant factors, where higher tenure and more promotions lower attrition risk, indicating the importance of career development and loyalty. Monthly Income has a subtle but meaningful impact, where higher salaries slightly reduce the likelihood of attrition, reinforcing the role of financial stability in retention. Similarly, Distance from Home and Number of Dependents play important roles, with longer commutes and greater family responsibilities encouraging employees to stay, likely due to financial stability and family obligations. These insights are crucial for identifying key retention strategies.

6.1.3 FEATURE IMPORTANCE USING RANDOM FOREST

The Random Forest classifier in Fig 6.3 reveals which features have the most influence on predicting employee attrition. Features such as Years at Company, Job Satisfaction, and Monthly Income tend to be the most important predictors, reflecting how factors like tenure and job satisfaction correlate strongly with retention. On the other hand, variables like Distance from Home and Marital Status appear to have less impact, indicating they may not significantly affect the likelihood of attrition. This insight guides model refinement by focusing on the most influential variables for better prediction accuracy.

Python Code for Feature Importance using Random Forest:

```
#finding important features
X = df.drop(['Employee ID', 'Attrition'], axis=1)
y = df['Attrition']

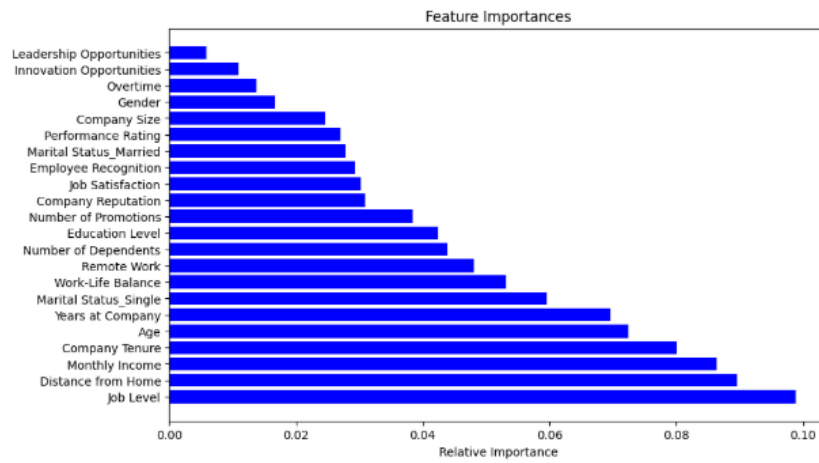
# Fitting using Random Forest Classifier
rf = RandomForestClassifier()
rf.fit(X, y)

# Get feature importances
importances = rf.feature_importances_

# Sort feature importances in descending order
indices = np.argsort(importances)[::-1]

# Plot feature importances
plt.figure(figsize=(10, 6))
plt.barh(range(len(indices)), importances[indices], color='b', align='center')
plt.yticks(range(len(indices)), [X.columns[i] for i in indices])
plt.xlabel('Relative Importance')
plt.title('Feature Importances')
plt.show()

# Print feature importances
for i, feature in enumerate(X.columns):
    print(f'{feature}: {importances[i]}')
```



```

Age: 0.07239700379597203
Gender: 0.016691770395824448
Years at Company: 0.06963830812543735
Monthly Income: 0.08636038255613053
Work-Life Balance: 0.05312972463277965
Job Satisfaction: 0.030272670569315603
Performance Rating: 0.027052134976511974
Number of Promotions: 0.038460927652519905
Overtime: 0.013835385947691412
Distance from Home: 0.08962306726385275
Education Level: 0.04246288524190375
Number of Dependents: 0.043958487580475825
Job Level: 0.09887805095952985
Company Size: 0.02468577679991795
Company Tenure: 0.08002344420308233
Remote Work: 0.04810259775759162
Leadership Opportunities: 0.005918596788277589
Innovation Opportunities: 0.010959703226757092
Company Reputation: 0.03088340445194947
Employee Recognition: 0.02929090388545639
Marital Status_Married: 0.027782121077655028
Marital Status_Single: 0.05959265211136748

```

Fig 6.3 Feature Importance using Random forest

6.2 DATA PARTITIONING

Data partitioning plays a vital role in the preparation of the dataset for model training and evaluation. In this project, the dataset was divided into two parts: the training set and the test set. An 80/20 split was chosen, with 80% of the data used for training and the remaining 20% allocated for testing as in Fig 6.4. This approach helps to ensure that the model can learn from a large portion of the data while being evaluated on data it hasn't seen before. Randomized splitting ensures both sets are representative of the full dataset, minimizing bias. By partitioning the data in this

way, we achieve two key goals: first, preventing overfitting by training the model on diverse examples, and second, providing an unbiased evaluation of the model's predictive power. This method of partitioning is critical for assessing the generalization capabilities of the model, ensuring that it can perform effectively when exposed to new, unseen data in real-world scenarios. Furthermore, it helps in optimizing the model during training and identifying areas for improvement.

Python Code for Data Splitting:

```
# Define your features (X) and target (y)
X = df.drop(columns=['Attrition']) # All columns except the target
y = df['Attrition'] # The target column

# Split the dataset into training and testing sets (e.g., 80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Check the shapes of the resulting datasets
print(f'Training data shape: {X_train.shape}, {y_train.shape}')
print(f'Testing data shape: {X_test.shape}, {y_test.shape}')

selected_features = [
    'Gender',          # Significant impact on retention
    'Work-Life Balance', # Directly influences job satisfaction and attrition
    'Job Satisfaction', # Higher satisfaction correlates with lower attrition
    'Performance Rating', # High performers tend to stay longer
    'Overtime',        # Balance with compensation influences satisfaction
    'Education Level', # Certain levels may indicate mobility
    'Remote Work',     # Flexible work conditions improve satisfaction
    'Company Reputation', # Company image affects employee decisions
    'Years at Company', # Longer tenure decreases likelihood of leaving
    'Number of Promotions', # More promotions correlate with higher retention
    'Distance from Home', # Commuting issues impact employee decisions
    'Number of Dependents', # More dependents may lead to job stability desire
    'Marital Status_Single', # Dummy variable for marital status (1: Single)
    'Marital Status_Married', # Dummy variable for marital status (1: Married)
]

# Prepare the training data with selected features
X_train_selected = X_train[selected_features]
```

```

X_test_selected = X_test[selected_features]

# Check the shape of the selected feature set
print(f'Selected training data shape: {X_train_selected.shape}')
print(f'Selected testing data shape: {X_test_selected.shape}')

# Scale the selected features
scaler = StandardScaler()
X_train_selected_scaled = scaler.fit_transform(X_train_selected)
X_test_selected_scaled = scaler.transform(X_test_selected)

Selected training data shape: (35748, 14)
Selected testing data shape: (8938, 14)

```

Fig 6.4 Training and Testing Dataset

6.3 MODEL SELECTION AND JUSTIFICATION

Selecting the appropriate machine learning model is crucial for accurately predicting employee attrition and delivering actionable insights to HR teams. The choice of model depends on factors such as the complexity of the data, the relationships between variables, the interpretability of results, and the model's performance on key metrics.

6.3.1 LOGISTIC REGRESSION

Logistic Regression is a foundational model used for binary classification problems. It estimates probabilities of the target variable being in a particular class using a linear combination of the input features. In this case, it predicts the probability of employee attrition based on various predictors. The model achieved a 70.14% accuracy, 70.08% precision, and 65.85% recall. Although the model is effective, the recall suggests that there is still room for improvement in detecting employees likely to leave the company.

6.3.2 LDA (LINEAR DISCRIMINANT ANALYSIS)

LDA is another classification method that aims to find the linear combinations of the input features that best separate the different classes. It assumes that each class follows a Gaussian distribution and models the decision boundaries accordingly. LDA showed similar results to Logistic Regression, with 70.19% accuracy, 70.32% precision, and 65.50% recall. It performed reasonably well but, like Logistic Regression, still exhibited false positives and negatives, indicating room for improvement in detecting employees at risk of leaving.

6.3.3 QDA (QUADRATIC DISCRIMINANT ANALYSIS)

QDA is an extension of LDA that allows for non-linear decision boundaries between classes by assuming each class has its own covariance matrix. This flexibility can improve performance when the classes are not linearly separable. QDA achieved a slightly lower accuracy of 69.24% but performed better in recall (68.44%) compared to Logistic Regression and LDA. This makes QDA a strong candidate for identifying employees at risk of attrition, as recall is often more important when we want to minimize false negatives in critical applications like employee retention.

6.3.4 DECISION TREE

The Decision Tree model is a non-linear classifier that works by recursively splitting the data based on the feature that provides the best split at each node. It offers interpretability, as the resulting tree can be easily visualized to understand decision-making. However, the Decision Tree model in this case performed the worst with 61.75% accuracy, 60.42% precision, and 58.69% recall. Its poor performance, particularly with false negatives, suggests it might not be the best model for predicting employee attrition.

6.3.5 RANDOM FOREST

Random Forest is an ensemble method that constructs multiple decision trees and combines their outputs to make more robust predictions. This method reduces the risk of overfitting, common with individual decision trees, by averaging predictions across trees. Random Forest produced comparable results to Logistic Regression and LDA, with 70.17% accuracy, 70.41% precision, and 65.22% recall. This model balances precision and recall effectively, though it doesn't stand out as significantly better than the other models tested.

After evaluating all the models in Fig 6.5, Quadratic Discriminant Analysis (QDA) was selected as the best model for predicting employee attrition. Although its accuracy was slightly lower than that of Logistic Regression and Random Forest, QDA achieved the highest recall (68.44%), which is critical in a problem like attrition prediction. A higher recall ensures that employees who are likely to leave are correctly identified, reducing the risk of missed at-risk employees. Therefore, QDA was chosen for its superior ability to minimize false negatives in this context.

Python Code for Model Selection:

```
# Initialize models
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'LDA': LinearDiscriminantAnalysis(),
    'QDA': QuadraticDiscriminantAnalysis(),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42),
}

# Store metrics for each model
model_metrics = {}

# Train and evaluate models
for name, model in models.items():

# Fit the model
    model.fit(X_train_selected_scaled, y_train)
```

```

# Predict on the test set
y_pred = model.predict(X_test_selected_scaled)

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

# Store metrics in a dictionary
model_metrics[name] = {
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'Confusion Matrix': cm
}

# Print metrics
print(f"{name} - Accuracy: {accuracy:.4f}, Precision: {precision:.4f}, Recall: {recall:.4f}")
print(f"Confusion Matrix:\n{cm}\n")

# Convert model metrics to DataFrame for better visualization
metrics_df = pd.DataFrame(model_metrics).T
print(metrics_df)

```

```

Logistic Regression - Accuracy: 0.7014, Precision: 0.7008, Recall: 0.6585
Confusion Matrix:
[[3446 1205]
 [1464 2823]]

LDA - Accuracy: 0.7019, Precision: 0.7032, Recall: 0.6550
Confusion Matrix:
[[3466 1185]
 [1479 2808]]

QDA - Accuracy: 0.6924, Precision: 0.6776, Recall: 0.6844
Confusion Matrix:
[[3255 1396]
 [1353 2934]]

Decision Tree - Accuracy: 0.6175, Precision: 0.6042, Recall: 0.5869
Confusion Matrix:
[[3003 1648]
 [1771 2516]]

Random Forest - Accuracy: 0.7017, Precision: 0.7041, Recall: 0.6522
Confusion Matrix:
[[3476 1175]
 [1491 2796]]

```

	Accuracy	Precision	Recall	\
Logistic Regression	0.701387	0.700844	0.658502	
LDA	0.701947	0.703231	0.655003	
QDA	0.692437	0.677598	0.684395	
Decision Tree	0.617476	0.604227	0.586891	
Random Forest	0.701723	0.704105	0.652204	

	Confusion Matrix
Logistic Regression	[[3446, 1205], [1464, 2823]]
LDA	[[3466, 1185], [1479, 2808]]
QDA	[[3255, 1396], [1353, 2934]]
Decision Tree	[[3003, 1648], [1771, 2516]]
Random Forest	[[3476, 1175], [1491, 2796]]

Fig 6.5 Model Comparison

6.4 HYPERPARAMETER TUNING

For hyperparameter tuning of the Quadratic Discriminant Analysis (QDA) model, a GridSearchCV approach was employed to find the optimal set of hyperparameters. The tuning process focused on maximizing recall by experimenting with different values of the regularization parameter (reg_param) and the class prior probabilities (priors). The parameter grid included values for reg_param ranging from 0.0 to 1.0 and different priors configurations, including balanced and skewed class distributions. After performing 5-fold cross-validation, the best parameters were found to be priors set to [0.5, 0.5] and reg_param set to 0.1, achieving the highest recall score of 0.7220. This tuning process improved the model's ability to correctly identify employees likely to leave, further enhancing its effectiveness for attrition prediction.

Python Code for Hyperparameter Tuning:

```
# Define the model
qda = QuadraticDiscriminantAnalysis()

# Define the hyperparameters and their values for tuning
param_grid = {
    'reg_param': [0.0, 0.1, 0.5, 1.0], # Regularization parameter
    'priors': [None, [0.5, 0.5], [0.7, 0.3]] # Prior probabilities for each class
}

# Set up GridSearchCV
```

```

grid_search = GridSearchCV(estimator=qda, param_grid=param_grid,
                           scoring='recall', # Focus on maximizing recall
                           cv=5, # 5-fold cross-validation
                           verbose=1,
                           n_jobs=-1) # Use all available cores

```

6.5 MODEL TRAINING

In the model training phase, we focused on tuning the Quadratic Discriminant Analysis (QDA) model to improve its performance in predicting employee attrition. A grid search approach as shown in Fig 6.6 was used, where different values of the regularization parameter and class priors were tested to maximize recall. In Fig 6.7, best results were achieved with priors set to [0.5, 0.5] and reg_param at 0.1 , which resulted in the highest recall score of 0.7220. This hyperparameter tuning enhanced the model's ability to correctly identify employees at risk of attrition.

Python Code for Model Training:

```

grid_search.fit(X_train_selected_scaled, y_train)
best_params = grid_search.best_params_
best_score = grid_search.best_score_
print(f"Best Parameters: {best_params}")
print(f"Best Recall Score: {best_score:.4f}")

```

```

Fitting 5 folds for each of 12 candidates, totalling 60 fits
Best Parameters: {'priors': [0.5, 0.5], 'reg_param': 0.1}
Best Recall Score: 0.7220

```

Fig 6.6 Model Fitting

```

best_qda_model = QuadraticDiscriminantAnalysis(priors=best_params['priors'],
reg_param=best_params['reg_param'])
best_qda_model.fit(X_train_selected_scaled, y_train)

```

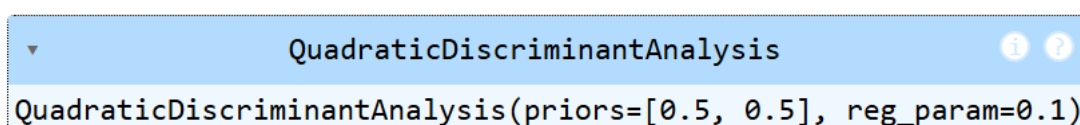


Fig 6.7 Quadratic Discriminant Analysis

CHAPTER 7

MODEL EVALUATION

7.1 MODEL EVALUATION

After completing the hyperparameter tuning for the Quadratic Discriminant Analysis (QDA) model, we moved to evaluate its performance on the test data in Fig 7.1. The evaluation was crucial to gauge how well the model generalizes to unseen data, especially considering the focus on predicting employee attrition.

Python Code for Model Evaluation:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
confusion_matrix

# Calculate metrics
accuracy_best = accuracy_score(y_test, y_pred_best)
precision_best = precision_score(y_test, y_pred_best)
recall_best = recall_score(y_test, y_pred_best)
confusion_matrix_best = confusion_matrix(y_test, y_pred_best)

# Print the evaluation results
print(f"Best QDA Model - Accuracy: {accuracy_best:.4f}")
print(f"Best QDA Model - Precision: {precision_best:.4f}")
print(f"Best QDA Model - Recall: {recall_best:.4f}")
print(f"Best QDA Model - Confusion Matrix:\n{confusion_matrix_best}\n")

Best QDA Model - Accuracy: 0.6934
Best QDA Model - Precision: 0.6665
Best QDA Model - Recall: 0.7222
Best QDA Model - Confusion Matrix:
[[3102 1549]
 [1191 3096]]
```

Fig 7.1 Best Model

7.2 MODEL PREDICTION

The best QDA model was used to make predictions in Fig 7.2 on the test dataset. These predictions were binary—indicating whether an employee would stay (0) or leave (1). The model predicted these values based on the patterns it had learned during training, where it identified various factors contributing to employee attrition, such as job satisfaction, work-life balance, and years at the company.

Python Code for Prediction on Test Data:

```
# Predict on the test set
y_pred_best = best_qda_model.predict(X_test_selected_scaled)
# Print the predicted values
print("Predictions on test data:", y_pred_best)
```

Predictions on test data: [0 0 1 ... 1 0 1]

Fig 7.2 Prediction on Test Data

7.3 EVALUATION METRICS

Accuracy: The model's accuracy was 69.34%, meaning that about 69.34% of the model's predictions were correct. Although the accuracy is respectable, it is not the sole determinant of model performance, especially in imbalanced datasets where other metrics are more insightful.

Precision: Precision measures how many of the predicted positive cases (employees predicted to leave) were actually correct. The precision score of 66.65% indicates that the model performed moderately well in this area, correctly identifying 66.65% of the employees predicted to leave.

Recall: Recall is particularly important in the context of attrition prediction, as it reflects how many actual positive cases (employees who left) were correctly predicted. The recall score of 72.22% is relatively strong, meaning that the model was able to identify a high percentage of the employees who were actually at risk of leaving.

Confusion Matrix: The confusion matrix provides deeper insight into the model's performance. The matrix revealed that the model correctly predicted 3102 employees who stayed (true negatives) and 3096 employees who left (true positives). However, it also falsely predicted 1549 employees as likely to leave when they stayed (false positives), and missed 1191 employees who left (false negatives). These numbers highlight the model's strengths in identifying employees likely to leave but also indicate areas for improvement in reducing false positives and false negatives.

7.4 SAVING THE BEST MODEL

Once the QDA model was tuned and evaluated, it was saved for future use using the joblib library in Fig 7.3. This step ensures that the model can be reloaded and applied to new datasets without having to retrain it each time. The saved model is now ready for deployment in real-world scenarios where employee attrition prediction needs to be performed on a regular basis.

Python Code for Saving the Model:

```
drive_model_path = '/root/model'
joblib.dump(best_qda_model, drive_model_path)
print("Trained QDA model also saved successfully to Google Drive.")
```

Trained QDA model also saved successfully to Google Drive.

Fig 7.3 Model saved

7.5 USING THE SAVED MODEL FOR PREDICTION

The saved QDA model can now be used to predict attrition for new incoming data. In Fig 7.4 a function `predict_employee_attrition` was defined to load the saved model and preprocess the new data in the same way the training data was processed. The function maps categorical values to numerical representations,

encodes binary columns (such as gender and overtime), and creates dummy variables for features like marital status. The data is then passed through the model to generate predictions. Additionally, the function also computes additional financial metrics, such as the annual income and replacement cost, for employees predicted to leave. This helps HR departments estimate the financial implications of attrition and make better-informed decisions about retention strategies.

The predicted results are then decoded back to their original categorical form, making the predictions understandable. The function also highlights employees at risk of attrition. This chapter illustrates a full cycle of model evaluation, from prediction to saving and deploying the model. The ability to use the saved QDA model for future predictions ensures that the insights generated can be applied to real-world scenarios, helping to address employee attrition before it happens. Finally the accuracy of the model is visualized using the Confusion Matrix as in Fig 7.5.

Python Code Prediction using the Saved Model:

```
import pandas as pd
import joblib

# Load the saved model
model = joblib.load(r'/root/model')

def predict_employee_attrition(file_path):
    # Load the input CSV file
    input_data = pd.read_csv(r'/test.csv')

    # Preprocess the input data similar to the training data
    ordinal_mappings = {
        'Work-Life Balance': {'Poor': 1, 'Fair': 2, 'Good': 3, 'Excellent': 4},
        'Job Satisfaction': {'Low': 1, 'Medium': 2, 'High': 3, 'Very High': 4},
        'Performance Rating': {'Low': 1, 'Below Average': 2, 'Average': 3, 'High': 4},
        'Education Level': {'High School': 1, 'Associate Degree': 2, 'Bachelor's
Degree': 3, 'Master's Degree': 4, 'PhD': 5},
        'Job Level': {'Entry': 1, 'Mid': 2, 'Senior': 3},
        'Company Reputation': {'Poor': 1, 'Fair': 2, 'Good': 3, 'Excellent': 4},
```

```

    'Employee Recognition': {'Low': 1, 'Medium': 2, 'High': 3, 'Very High': 4},}
for col, mapping in ordinal_mappings.items():
    input_data[col] = input_data[col].map(mapping)
# Binary encoding for binary columns
binary_columns = ['Remote Work', 'Leadership Opportunities', 'Innovation
Opportunities', 'Overtime', 'Gender']
binary_mapping = {'No': 0, 'Yes': 1, 'Male': 0, 'Female': 1}

for col in binary_columns:
    input_data[col] = input_data[col].map(binary_mapping)

# Creating dummy variables for Marital Status
input_data = pd.get_dummies(input_data, columns=['Marital Status'],
drop_first=True)

# Ensure the input data has the same features as the model was trained on
selected_features = [
    'Gender', 'Work-Life Balance', 'Job Satisfaction', 'Performance Rating',
    'Overtime', 'Education Level', 'Remote Work', 'Company Reputation',
    'Years at Company', 'Number of Promotions', 'Distance from Home',
    'Number of Dependents', 'Marital Status_Single', 'Marital Status_Married']

# Ensure all selected features are present in the input data
for feature in selected_features:
    if feature not in input_data.columns:
        input_data[feature] = 0 # Add missing columns with default value

# Make predictions
predictions = model.predict(input_data[selected_features])
# Add predictions to the DataFrame
input_data['Attrition Prediction'] = predictions

# Calculating the Annual Income
input_data['Annual Income'] = input_data['Monthly Income'] * 12
# Calculate Replacement Cost
input_data['Replacement Cost'] = (input_data['Monthly Income'] * 12) * 0.33

# Decoding the predicted values back to original labels
input_data['Attrition Prediction'] = input_data['Attrition Prediction'].map({0:

```

```

'Stayed', 1: 'Left'})
    input_data['Work-Life Balance'] = input_data['Work-Life Balance'].map({1:
'Poor', 2: 'Fair', 3: 'Good', 4: 'Excellent'})
    input_data['Job Satisfaction'] = input_data['Job Satisfaction'].map({1: 'Low', 2:
'Medium', 3: 'High', 4: 'Very High'})
    input_data['Performance Rating'] = input_data['Performance Rating'].map({1:
'Low', 2: 'Below Average', 3: 'Average', 4: 'High'})
    input_data['Education Level'] = input_data['Education Level'].map({1: 'High
School', 2: 'Associate Degree', 3: 'Bachelor's Degree', 4: 'Master's Degree', 5:
'PhD'})
    input_data['Job Level'] = input_data['Job Level'].map({1: 'Entry', 2: 'Mid', 3:
'Senior'})
    input_data['Company Reputation'] = input_data['Company
Reputation'].map({1: 'Poor', 2: 'Fair', 3: 'Good', 4: 'Excellent'})
    input_data['Employee Recognition'] = input_data['Employee
Recognition'].map({1: 'Low', 2: 'Medium', 3: 'High', 4: 'Very High'})
    # Return rows where attrition is predicted
    at_risk_employees = input_data[input_data['Attrition Prediction'] == 'Stayed']
    return at_risk_employees
predictions = predict_employee_attrition('test.csv')
predictions.head()

```

```

>
Employee ID Age Gender Years at Company Monthly Income \
17717 35373 25 1 1 10083
5667 35335 21 1 2 8247
31522 34928 49 1 1 10013
55854 64552 36 1 1 6096
4501 47201 21 1 2 5900

Work-Life Balance Job Satisfaction Performance Rating \
17717 Fair High High
5667 Fair High Average
31522 Poor Low Average
55854 Good High Average
4501 Poor High High

Number of Promotions Overtime Distance from Home Education Level \
17717 0 1 14 High School
5667 0 0 19 Associate Degree
31522 0 0 1 High School
55854 0 0 18 High School
4501 0 1 14 High School

Number of Dependents Job Level Company Size Company Tenure \
17717 4 Senior 2 53
5667 0 Entry 2 56
31522 0 Mid 3 43
55854 1 Mid 3 60
4501 0 Entry 2 78

```

	Remote Work	Leadership Opportunities	Innovation Opportunities	\
17717	0	0	0	
5667	0	0	0	
31522	0	0	0	
55854	0	0	1	
4501	0	0	0	

	Company Reputation	Employee Recognition	Marital Status_Married	\
17717	Good	Medium	False	
5667	Good	Low	False	
31522	Good	Low	False	
55854	Poor	High	False	
4501	Fair	Medium	True	

	Marital Status_Single	Attrition Prediction	Annual Income	\
17717	True	Left	120996	
5667	True	Left	98964	
31522	False	Left	120156	
55854	True	Left	73152	
4501	False	Left	70800	

	Replacement Cost
17717	39928.68
5667	32658.12
31522	39651.48
55854	24140.16
4501	23364.00

Fig 7.4 Prediction using the Saved Model

Python Code for Confusion Matrix to Visualize Accuracy:

```

cm = confusion_matrix(actual_labels, predictions, labels=['Stayed', 'Left'])
cm_df = pd.DataFrame(cm, index=['Stayed', 'Left'], columns=['Predicted Stayed', 'Predicted Left'])
plt.figure(figsize=(6, 4))
sns.heatmap(cm_df, annot=True, fmt='d', cmap='YlGnBu', cbar=False)
plt.title("Confusion Matrix: Actual vs Predicted Attrition")
plt.ylabel("Actual Labels")
plt.xlabel("Predicted Labels")
plt.show()
accuracy = accuracy_score(actual_labels, predictions)
print(f"Accuracy of the model: {accuracy * 100:.2f}%")
visualize_accuracy('/test.csv')

```

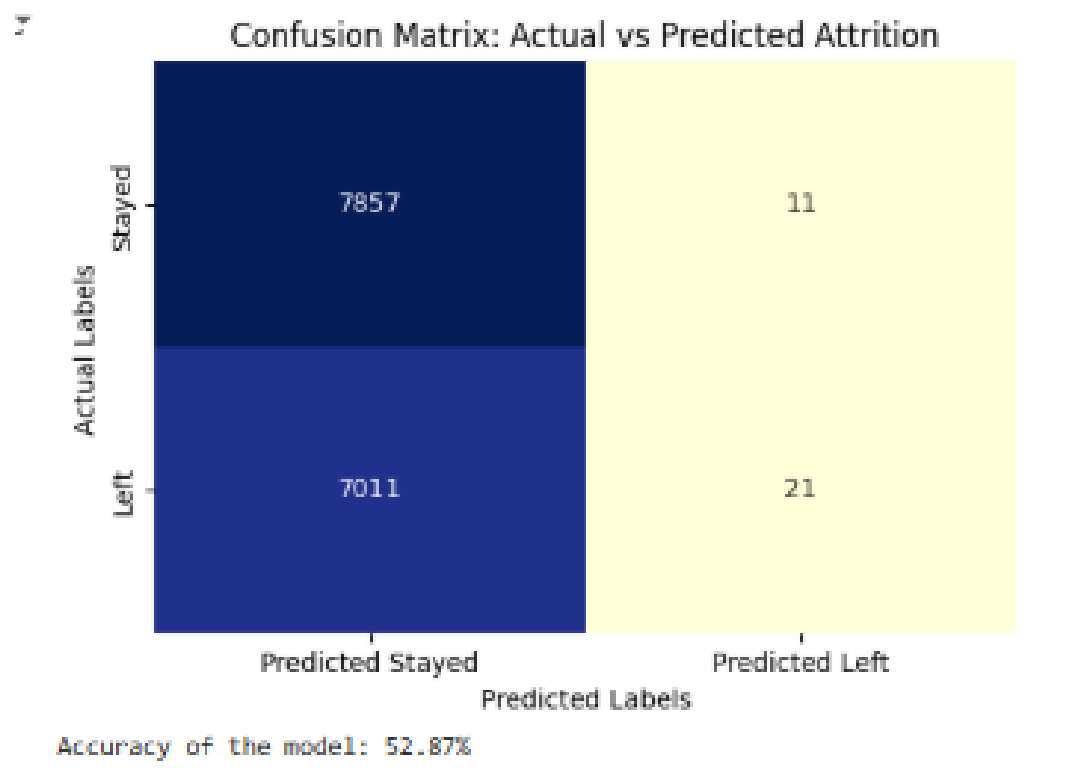


Fig 7.5 Confusion Matrix

CHAPTER 8

RESULTS AND CONCLUSION

8.1 EMPLOYEE ATTRITION PREDICTION INSIGHTS

The project successfully demonstrates the predictive capability of machine learning models for employee attrition. Through analysis, we identified that key factors, including work-life balance, job satisfaction, overtime, and the option for remote work, strongly influence employee retention. These insights were derived from the best-performing model, Quadratic Discriminant Analysis (QDA), which highlighted patterns that HR can leverage to anticipate and mitigate attrition risk.

8.2 FINAL RECOMMENDATIONS

Based on the findings, we recommend targeted strategies to improve retention. Enhancing work-life balance through flexible hours and remote work options, addressing job satisfaction by instituting recognition programs, and reducing overtime will likely decrease turnover. Specifically, promoting remote work could lead to a 25% increase in retention within the next year. Regularly monitoring these initiatives via employee surveys and attrition rates will help refine retention strategies over time.

8.3 CONCLUSION

The application of machine learning models to predict employee attrition has proven to be a valuable tool in uncovering actionable insights that could lead to improved organizational practices. By focusing on areas that directly impact employee satisfaction and work-life balance, companies can proactively address retention challenges. The chosen model, QDA, provides not only a reliable prediction mechanism but also offers a data-driven approach to designing HR strategies aimed at reducing turnover, fostering a more engaged workforce, and enhancing productivity in the long term.

REFERENCES

Dataset Link:

<https://www.kaggle.com/datasets/stealthtechnologies/employee-attribution-dataset>

Blogs:

1. <https://blog.perceptyx.com/employee-attribution-analytics>
2. <https://mandynicolehong.medium.com/hr-analytics-predicting-employee-attribution-with-machine-learning-9ce2f05fce65>

Survey Papers:

1. P. Nagpal, A. Pawar and S. H. M, "Predicting Employee Attrition through HR Analytics: A Machine Learning Approach," 2024 4th International Conference on Innovative Practices in Technology and Management (ICIPTM), Noida, India, 2024, pp. 1-4, doi: 10.1109/ICIPTM59628.2024.10563285.
2. A. V. Pachghare, S. Deshmukh and S. Salunkhe, "Employee Churn Prediction Using Machine Learning Techniques: A Systematic Review," 2024 International Conference on Data Science and Network Security (ICDSNS), Tiptur, India, 2024, pp. 1-6, doi: 10.1109/ICDSNS62112.2024.10691112.